

FORCEBIT TICKETING.

Een support ticketing applicatie met een gelaagde architectuur.

Alex Delvoye.

GraduaatsProef Programmeren 2025–2026.



Scriptie voorgedragen tot het bekomen van de graad van
Graduaat in het Programmeren

Promotor: Emlin Oguz Inci

Co-promotor: Sacha De Maere

Instelling: Forcebit

Academiejaar: 2025–2026

Tweede examenperiode

Departement IT en Digitale Innovatie .

**HO
GENT**

Woord vooraf

Deze graduaatsproef is voor mij een kans geweest om de leerstof van de opleiding Graduaat Programmeren toe te passen in een project dat dicht bij een realistische werksituatie ligt. Met Forcebit Ticketing wilde ik de geziene leerstof herhalen en combineren in een oplossing voor een concreet probleem bij Forcebit: communicatie met klanten stond verspreid over telefoon en e-mail, waardoor opvolging minder overzichtelijk was. De applicatie brengt tickets, gesprekken, statussen en bijlagen samen in één centrale workflow.

Technisch bestaat het project uit een gelaagde backend in C# en ASP.NET Core, gekoppeld aan een MySQL-databank. Daarin gebruikte ik onder andere Dependency Injection, het Repository Pattern, een Service Layer, DTO-mapping en het Options Pattern. Ook de frontend is gestructureerd opgebouwd met aparte schermen, eigen hooks, API-modules, formulieren, componenten, validatieschema's en hulpfuncties, zodat de code herbruikbaar en beter onderhoudbaar blijft.

Tijdens dit project heb ik ook nieuwe onderdelen leren toepassen, zoals een mail-service, het uploaden van bijlagen, previews voor afbeeldingen, authenticatie met rollen en een duidelijke ticketworkflow. Daardoor voelde deze graduaatsproef voor mij als een passende afsluiter van de opleiding: een herhaling van wat ik geleerd heb, aangevuld met technieken die dichter bij echte softwareontwikkeling liggen.

Graag wil ik mijn mentor Emlin Oguz Inci bedanken voor het opvolgen van deze graduaatsproef en voor zijn feedback over de functionaliteiten, structuur en onderhoudbaarheid van het project. Zijn opmerkingen hebben mij geholpen om niet alleen naar werkende code te kijken, maar ook naar gebruiksgemak, onderhoudbaarheid en duidelijke uitleg.

Samenvatting

Forcebit Ticketing is een webapplicatie voor het opvolgen van supporttickets. Het project bestaat uit een ASP.NET Core backend, een MySQL-databank en een Expo React Native frontend. Klanten kunnen zich registreren, inloggen, tickets aanmaken, antwoorden sturen en bijlagen uploaden. Administrators beheren tickets via een dashboard met zoeken, filters, statusgroepen en paginatie.

De aanleiding voor dit project is het probleem dat klantcommunicatie bij Forcebit verspreid stond over telefoon en e-mail. Daardoor werd het moeilijker om supportvragen centraal op te volgen. De applicatie brengt deze communicatie samen in tickets met vaste metadata zoals titel, categorie, onderwerp en status. De volledige conversatie blijft per ticket bewaard, inclusief geüploade bijlagen. Afbeeldingen kunnen rechtstreeks bekeken worden, terwijl andere bestanden als download beschikbaar blijven.

De centrale onderzoeksvraag is hoe een onderhoudbare applicatie gebouwd kan worden waarin klant en administrator een duidelijke workflow volgen terwijl de functionaliteit blijft groeien. Hiervoor werd het project opgebouwd in lagen. De API-laag behandelt HTTP-verkeer en authenticatieclaims, de service-laag voert de use cases uit, de Persistence-laag beheert Entity Framework Core en MySQL, en de Domain-laag bevat entiteiten en regels rond tickets, berichten en bijlagen.

Tijdens de uitwerking lag de nadruk op Single Responsibility. Controllers verwerken geen ticketregels, services sturen workflows aan en repositories vormen de grens met de databank. In de frontend werd dezelfde scheiding toegepast met aparte schermen, eigen hooks, API-modules, formulieren, componenten, validatieschema's en hulpfuncties. Zo blijven zoeken, groeperen, pagineren, foutafhandeling en bijlagelogica herbruikbaar.

Het resultaat is een werkend proof of concept van Forcebit Ticketing met een realistische supportworkflow. Tickets starten als **New**, worden **Open** wanneer support antwoordt en kunnen **Closed** worden wanneer het probleem opgelost is. Wanneer een gesloten ticket opnieuw geopend wordt, gaat het terug naar **Open** en niet naar **New**. Daarnaast krijgen klanten e-mailnotificaties wanneer een administrator een antwoord plaatst of de status van een ticket wijzigt.

Belangrijke toekomstige verbeteringen zijn een uitgebreidere automatische testset, server-side paginatie en productiegerichte logging. Toch toont het project aan dat een gelaagde structuur en duidelijke verantwoordelijkheden helpen om een supportapplicatie overzichtelijk, onderhoudbaar en uitbreidbaar te maken.

Broncode: [alexdelvoye/GraduaatsProef_TicketingSystem.git](https://github.com/alexdelvoye/GraduaatsProef_TicketingSystem.git)

Summary

Forcebit Ticketing is a web application for managing support tickets. The project consists of an ASP.NET Core backend, a MySQL database, and an Expo React Native frontend. Clients can register, log in, create tickets, reply to conversations, and upload attachments. Administrators manage tickets through a dashboard with search, filters, status groups, and pagination.

The project was created to solve a practical communication problem at Forcebit. Client communication was spread across phone calls and e-mails, which made follow-up harder to manage centrally. The application stores this communication in tickets with fixed metadata such as title, category, subject, and status. The full conversation remains linked to the ticket, including uploaded attachments. Images can be previewed directly, while other files remain available as downloads.

The central research question is how to build a maintainable ticketing application in which clients and administrators follow a clear workflow while the feature set continues to grow. The backend was therefore divided into layers. The API layer handles HTTP traffic and authentication claims, the service layer executes use cases, the persistence layer manages Entity Framework Core and MySQL, and the domain layer contains entities and rules for tickets, messages, and attachments.

Single Responsibility was an important design guideline. Controllers do not contain ticket rules, services coordinate workflows, and repositories form the boundary with the database. The frontend follows a similar structure with separate screens, custom hooks, API modules, forms, components, validation schemas, and utility functions. This keeps search, grouping, pagination, error handling, and attachment logic reusable.

The result is a working proof of concept of Forcebit Ticketing with a realistic support workflow. Tickets start as **New**, become **Open** when support replies, and can be marked **Closed** when the issue is solved. When a closed ticket is reopened, it returns to **Open** instead of **New**. Clients also receive e-mail notifications when an administrator replies or changes the status of a ticket.

Important future improvements include a broader automated test suite, server-side pagination, and production-oriented logging. Even so, the project shows that layered structure and clear responsibilities help keep a support application organized, maintainable, and extendable.

Source code: [alexdelvoye/GraduaatsProef_TicketingSystem.git](https://github.com/alexdelvoye/GraduaatsProef_TicketingSystem.git)

Inhoudsopgave

Lijst van figuren	vii
Lijst van codefragmenten	viii
1 Inleiding	1
1.1 Probleemstelling	1
1.2 Onderzoeksvraag en doelstelling.	1
2 Stand van zaken	2
2.1 Ticketing als ondersteuning van supportprocessen	2
2.2 Web API en backendstructuur	2
2.3 Single Responsibility en lagen	2
2.4 Frontend state en herbruikbaarheid	3
2.5 Beveiliging en validatie	3
3 Methodologie	4
3.1 Aanpak in fasen	4
4 Functionele analyse	5
4.1 Gebruikersrollen.	5
4.1.1 Klant	5
4.1.2 Administrator	5
4.2 Ticketstatussen	6
4.3 Bijlagen	6
4.4 Notificaties	6
5 Backendarchitectuur en beveiliging	7
5.1 Gelaagde structuur.	7
5.2 Controllers en DTO's	8
5.3 Services als use cases	8
5.4 Domeinregels	9
5.5 Authenticatie en autorisatie	9
6 Databank en persistentie	10
6.1 Datamodel	10
6.2 DbContext, configuraties en repositories	11
6.3 Enumwaarden en relaties	11

7 Frontendarchitectuur en gebruikersinterface	12
7.1 Schermen, hooks en componenten	12
7.2 Dashboardlogica	13
7.3 Paginatie en hergebruik	13
7.4 Ticketgroepen en conversatie	14
8 Aanvullende functionaliteiten	15
8.1 E-mailnotificaties met Brevo	15
8.2 Bijlagen en uploadlimiet	15
9 Conclusie	16
Bibliografie	17

Lijst van figuren

2.1	Samenvatting stand van zaken	3
4.1	Klantenlijst in het adminoverzicht	6
5.1	Backendlagen	8
6.1	Databankschema	10
7.1	Frontendarchitectuur	12
7.2	Ticketfilters	13
7.3	Ticketconversatie	14

Lijst van codefragmenten

5.1	Statuscontrole in TicketRules	9
6.1	DbContext met configuraties	11
7.1	Generieke paginatiehook	13

1

Inleiding

Forcebit Ticketing vertrekt vanuit een concreet probleem bij Forcebit: supportcommunicatie met klanten stond vooral verspreid over telefoon en e-mail. Daardoor was het niet altijd duidelijk welke vragen nieuw waren, welke al behandeld werden en welke bijlagen bij welke vraag hoorden. De applicatie centraliseert die communicatie in tickets met een titel, categorie, onderwerp, status, conversatie en bijlagen.

1.1. Probleemstelling

Een ticketingsysteem brengt structuur in supportcommunicatie, maar alleen wanneer de regels duidelijk blijven. Rechten, antwoorden, statusovergangen en bijlagen mogen niet verspreid raken over controllers, schermen of losse hulpfuncties. De uitdaging is dus niet alleen een werkende applicatie bouwen, maar ook een codebase die onderhoudbaar blijft wanneer de functionaliteit groeit.

1.2. Onderzoeksvraag en doelstelling

Hoe kan een onderhoudbare ticketingapplicatie gebouwd worden waarin klant en administrator een duidelijke supportworkflow volgen, met aandacht voor Single Responsibility, gelaagde structuur en herbruikbare frontendlogica?

Het doel is een werkend proof of concept te bouwen waarin:

- klanten tickets kunnen aanmaken, beantwoorden, sluiten en heropenen;
- administrators tickets kunnen zoeken, filteren, beantwoorden en van status veranderen;
- backendlagen, DTO's, domeinregels, repositories, hooks en herbruikbare componenten de onderhoudbaarheid ondersteunen.

Daarna bespreekt het verslag achtereenvolgens theorie, aanpak, functionele analyse, backend, databank, frontend, aanvullende functionaliteiten en conclusie.

2

Stand van zaken

Dit hoofdstuk vat de concepten samen die nodig zijn om de gemaakte keuzes te plaatsen: ticketing, web API's, lagen, frontendstate en beveiliging.

2.1. Ticketing als ondersteuning van supportprocessen

Een ticketingsysteem houdt vragen, antwoorden en bijlagen centraal bij. Binnen IT service management sluit dat aan bij het opvolgen van incidenten en service requests, waarbij structuur en traceerbaarheid belangrijk zijn (AXELOS, [2019](#); ISO/IEC, [2018](#)). Ook Atlassian benadrukt dat tickets aanvragen samenbrengen, zodat opvolging niet over losse kanalen verspreid raakt (Atlassian, [2026](#)). Voor Forcebit Ticketing betekent dit dat de eerste beschrijving als eerste bericht in de conversatie wordt bewaard. Zo heeft elk ticket vanaf het begin dezelfde structuur.

2.2. Web API en backendstructuur

ASP.NET Core biedt controllers, dependency injection, middleware en configuratie voor web API's (Microsoft, [2026a](#)). Die onderdelen passen bij een backend die HTTP-verzoeken verwerkt, authenticatie controleert en services aanroept. Entity Framework Core wordt gebruikt als Object-Relational Mapper: C#-entiteiten worden via configuraties en migraties gekoppeld aan een relationele databank (Microsoft, [2026c](#)). Dat is bruikbaar voor tickets, berichten, bijlagen en gebruikers omdat deze duidelijke relaties hebben.

2.3. Single Responsibility en lagen

Single Responsibility betekent dat een klasse of module een duidelijke hoofdreden heeft om te veranderen. Robert C. Martin koppelt dit principe aan onderhoudbare software: code blijft beter aanpasbaar wanneer verantwoordelijkheden niet door

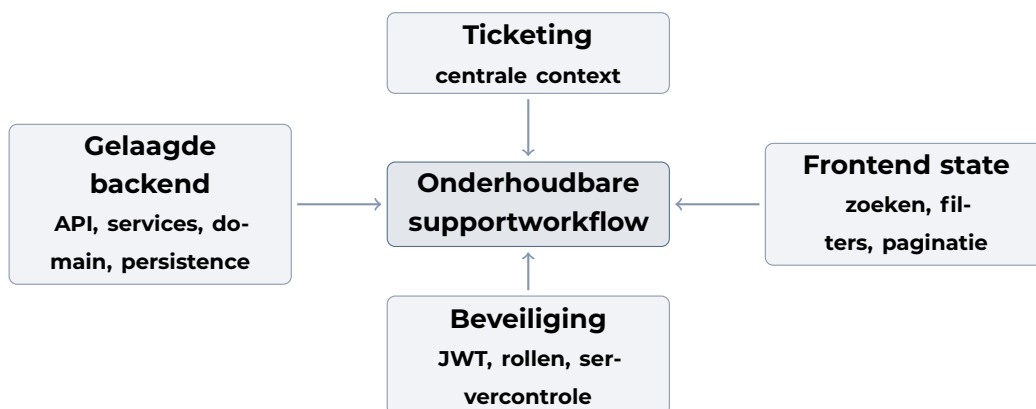
elkaar lopen (Martin, 2017). Daarom verdeelt een gelaagde structuur de logica. Controllers behandelen HTTP en geven gegevens door via Data Transfer Objects (DTO's), objecten die data over een grens transporteren (Fowler, 2003). Services voeren use cases uit: afgebakende gebruikersdoelen of systeemgedrag (Jacobson e.a., 1992). Repositories verzorgen databanktoegang en domeinregels bevatten businessregels zoals toegang en statusovergangen.

2.4. Frontend state en herbruikbaarheid

De frontend is gebouwd met Expo React Native. React Native werkt met componenten, terwijl Expo extra tooling aanbiedt voor ontwikkeling en webondersteuning (Expo, 2026; Meta Open Source, 2026). React Navigation verzorgt schermnavigatie en webroutes (React Navigation, 2026). In Forcebit Ticketing blijven ruwe API-resultaten de bron van waarheid. Zoekresultaten, filters, tellingen, groepen en paginatie worden daarvan afgeleid, zodat schermen niet elk hun eigen versie van dezelfde data bijhouden.

2.5. Beveiliging en validatie

Omdat tickets klantgegevens en bijlagen kunnen bevatten, mag de frontend niet de enige beveiligingslaag zijn. OWASP benadrukt dat toegangscontrole aan serverzijde moet worden afgedwongen (OWASP Foundation, 2026). Forcebit Ticketing gebruikt daarom JWT-authenticatie en rolgebaseerde autorisatie. Een JSON Web Token is een compact tokenformaat waarmee claims, zoals gebruikersidentiteit en rol, worden doorgegeven (Jones e.a., 2015). Frontendvalidatie helpt de gebruiker, maar backendvalidatie blijft noodzakelijk.



Figuur 2.1: Visuele samenvatting van de stand van zaken. De literatuurstudie toont hoe ticketing, een gelaagde backend, herbruikbare frontendlogica en beveiliging samen bijdragen aan een onderhoudbare supportworkflow.

3

Methodologie

Dit hoofdstuk beschrijft de aanpak op hoofdlijnen. Het oorspronkelijke plan vertrok van basis naar uitbreiding: requirements, databankontwerp, backend, authenticatie, ticketbeheer, berichten, frontend, adminschermpjes, uploads, e-mail, testen en documentatie. Tijdens de uitvoering werd dit bijgestuurd op basis van mentorfeedback, technische keuzes en onderhoudbaarheid.

3.1. Aanpak in fasen

- **Scope en analyse.** Eerst werden rollen, gegevens en workflows bepaald vanuit klant, administrator en systeem. Dit leverde de basis op voor gebruikers, tickets, berichten, bijlagen en de statusflow.
- **Technische basis.** Daarna werd het proof of concept opgebouwd met ASP.NET Core, Entity Framework Core en MySQL. De backend kreeg meteen lagen, DTO's, repositories, services, migraties, JWT-authenticatie, foutafhandeling en het Options Pattern (Microsoft, [2026d](#)).
- **Frontend en feedback.** De frontend werd gebouwd met Expo (Expo, [2026](#)). Formik en Yup ondersteunden formulieren en validatie (Formik, [2026](#); Quense, [2026](#)). Mentorfeedback leidde tot chatbubbels, afbeeldingspreview, uploadlimiet, responsieve lay-out, browsernavigatie, zoekvelden, filters en paginatie.
- **Controle en documentatie.** Ten slotte werd realistische testdata toegevoegd voor meerdere klanten, tickets en langere gesprekken. Filters, statusgroepen, paginatie, databankopstart, codecommentaar en README-bestanden werden gecontroleerd, zodat de applicatie makkelijker te onderhouden blijft.

4

Functionele analyse

Dit hoofdstuk beschrijft wat de applicatie functioneel moet doen. De focus ligt op de gebruikersrollen, de belangrijkste schermen en de ticketworkflow.

4.1. Gebruikersrollen

De applicatie heeft twee rollen: klant en administrator. Een klant gebruikt de applicatie om supportvragen te melden en op te volgen. Een administrator gebruikt de applicatie om tickets te beheren en te beantwoorden.

4.1.1. Klant

Een klant kan zich registreren, inloggen en een profiel beheren. Daarna kan de klant tickets aanmaken. Bij een ticket kiest de klant een categorie, onderwerp en titel. De beschrijving die de klant invult, wordt opgeslagen als eerste bericht in de conversatie. Later kan de klant antwoorden en bijlagen toevoegen zolang het ticket niet gesloten is.

Een klant kan een eigen ticket sluiten wanneer het probleem opgelost is. Als het probleem toch terugkomt, kan de klant het ticket opnieuw openen. Het ticket krijgt dan de status `Open`, niet `New`, omdat het geen nieuw ticket meer is.

4.1.2. Administrator

Een administrator kan alle tickets bekijken. Het adminoverzicht bevat een klantenlijst en een ticketlijst. Omdat die lijsten snel lang worden, zijn er zoekvelden, filters en paginatie toegevoegd. Administrators kunnen tickets beantwoorden en de status wijzigen naar `Open` of `Closed`.

4.2. Ticketstatussen

De ticketstatussen zijn bewust beperkt tot drie waarden:

- New: het ticket is aangemaakt, maar support heeft nog niet geantwoord.
- Open: de conversatie is actief of een gesloten ticket werd heropend.
- Closed: het ticket is afgehandeld en er kan niet meer geantwoord worden.

Deze namen zijn gekozen omdat ze duidelijker zijn voor de gebruiker. New toont dat een ticket nog wacht op de eerste supportreactie. Open toont dat er een actieve conversatie loopt of dat een gesloten ticket opnieuw behandeld wordt. Closed toont dat het probleem als opgelost beschouwd wordt.

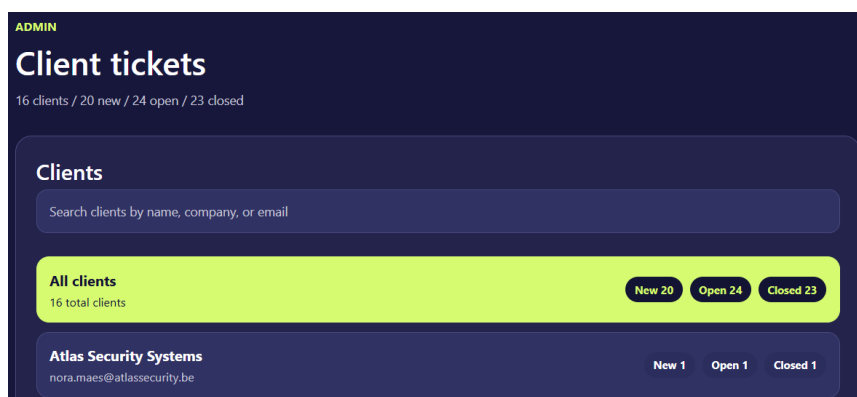
4.3. Bijlagen

Bijlagen kunnen toegevoegd worden bij ticketcreatie en bij antwoorden. Toegestane bestandstypes zijn PNG, JPG, JPEG, PDF en ZIP. De gebruiker ziet tijdens het selecteren hoeveel van de 20 MB uploadlimiet al gebruikt is. De backend controleert bestandstype en grootte opnieuw, zodat de frontend niet de enige controlelaag is.

Afbeeldingen krijgen een preview in de conversatie. Screenshots komen vaak voor bij supportvragen. De gebruiker hoeft een afbeelding dan niet eerst te downloaden om te zien wat erin staat.

4.4. Notificaties

Wanneer een administrator antwoordt of de status wijzigt, krijgt de klant een e-mailnotificatie. Klantacties sturen geen e-mail naar administrators. Administrators gebruiken het dashboard als werkqueue. Dit voorkomt dat een mailbox overspoeld wordt wanneer veel klanten antwoorden.



Figuur 4.1: Klantenlijst in het adminoverzicht met zoekveld en paginatie.

5

Backendarchitectuur en beveiliging

Dit hoofdstuk bespreekt de backend van Forcebit Ticketing. De nadruk ligt op de gelaagde structuur, de verdeling van verantwoordelijkheden en de manier waarop tickettoegang gecontroleerd wordt.

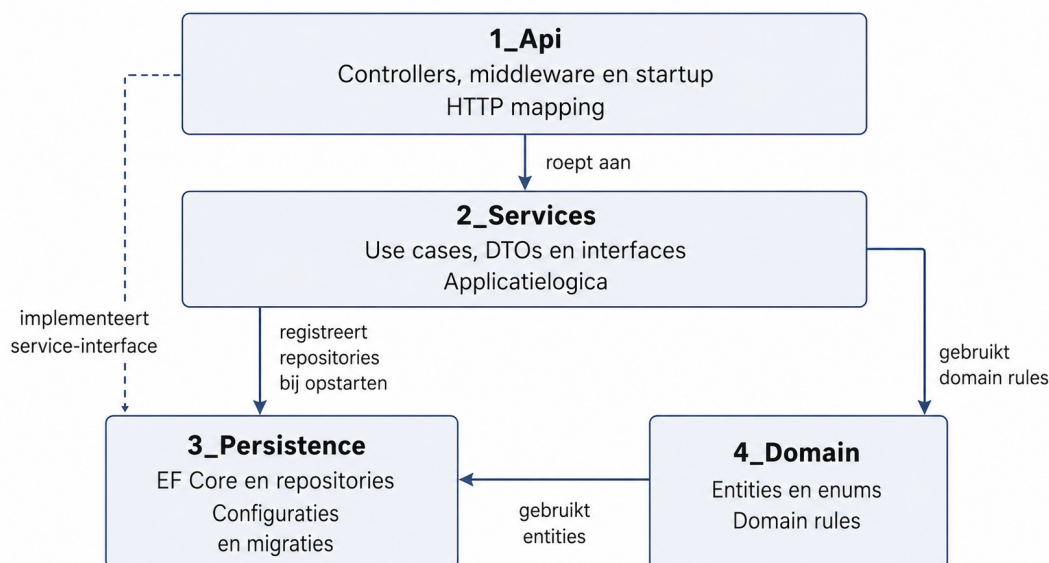
5.1. Gelaagde structuur

De backend bestaat uit vier projecten. Elke laag heeft een duidelijke hoofdtaak:

- **1_Api:** ontvangt HTTP-verzoeken, leest JWT-claims, mapt requestdata en registreert afhankelijkheden.
- **2_Services:** voert de use cases uit, gebruikt DTO's en bevat interfaces voor externe onderdelen zoals repositories en e-mail.
- **3_Persistence:** bevat Entity Framework Core, repositories, configuraties en migraties.
- **4_Domain:** bevat entiteiten, enums en domeinregels. Deze laag hangt niet af van de andere projectlagen.

De requestflow loopt meestal van API naar Services en daarna naar repositories. De API verwijst vooral naar de Persistence-laag om afhankelijkheden te registreren. Persistence implementeert de repositoryinterfaces uit Services. Daardoor kan de service-laag met abstracties werken, terwijl de concrete databankcode in de Persistence-laag blijft.

Deze opdeling ondersteunt Single Responsibility. Een controller hoeft niet te weten hoe een ticket uit MySQL wordt opgehaald. Een repository hoeft niet te beslissen of een klant een ticket mag sluiten. De Domain-laag blijft vrij van infrastructuurcode.



Figuur 5.1: Backendlagen en belangrijkste projectafhankelijkheden.

5.2. Controllers en DTO's

Controllers blijven bewust dun. Ze lezen routeparameters, requestdata, form-data en JWT-claims, maar voeren zelf geen ticketlogica uit. Daarna vertalen ze de binnekomende data naar DTO's en geven die door aan de service-laag.

DTO's vormen hier de grens tussen HTTP en applicatielogica. Bij uploads ontvangt ASP.NET Core bijvoorbeeld een bestand als `IFormFile`. De service-laag werkt niet met dat HTTP-type, maar met `FileUploadRequest`. Zo blijven use cases bruikbaar zonder rechtstreeks afhankelijk te zijn van controllerdetails.

5.3. Services als use cases

`TicketService` is de belangrijkste use-caseklasse. Deze service maakt tickets aan, laadt ticketdetails, voegt berichten toe, wijzigt statussen, bewaart bijlagen en beslist wanneer een e-mail nodig is. De service stuurt de workflow aan, maar laat HTTP-antwoorden over aan controllers en databankqueries aan repositories.

Dit maakt de service verantwoordelijk voor de volgorde van acties, niet voor elke technische uitvoering. Bestanden opslaan gebeurt via `IFileStorageService`, e-mail via `IEmailService` en databanktoegang via repositories. Daardoor kan een onderdeel later vervangen worden zonder de volledige ticketflow te herschrijven.

5.4. Domeinregels

Businessregels rond tickets staan in de Domain-laag. Daardoor kunnen services dezelfde regels gebruiken zonder ze te kopiëren. De statusregel hieronder toont dat `New` alleen door ticketcreatie mag ontstaan. Een gebruiker kan een bestaand ticket dus niet terug naar `New` zetten.

```
1  if (newStatus == TicketStatus.New)
2  {
3      return false;
4  }
5
6  if (userRole == UserRole.Admin)
7  {
8      return true;
9  }
10
11 if (userRole != UserRole.Client || ticket.ClientId != userId)
12 {
13     return false;
14 }
15
16 if (newStatus == TicketStatus.Closed)
17 {
18     return ticket.Status != TicketStatus.Closed;
19 }
20
21 return newStatus == TicketStatus.Open &&
22     ticket.Status == TicketStatus.Closed;
```

Codefragment 5.1: Statuscontrole in `TicketRules`.

De regel maakt ook het verschil tussen administrator en klant duidelijk. Een administrator mag een ticket naar `Open` of `Closed` zetten. Een klant mag alleen het eigen ticket sluiten of een gesloten ticket opnieuw openen. Daardoor blijft de betekenis van `New`, `Open` en `Closed` overal gelijk.

5.5. Authenticatie en autorisatie

Na login bevat het JWT-token de gebruikersidentiteit en rol. Controllers schermen routes af met rollen, terwijl de service-laag tickettoegang opnieuw controleert via domeinregels. Dat is nodig omdat een klant alleen eigen tickets mag zien, terwijl een administrator alle tickets mag beheren.

De frontend toont alleen acties die bij de rol passen, maar dat is niet voldoende als beveiliging. De backend blijft de beslissende laag. Ook wanneer iemand handmatig een API-aanroep probeert, controleert de service-laag opnieuw of de gebruiker de gevraagde actie mag uitvoeren.

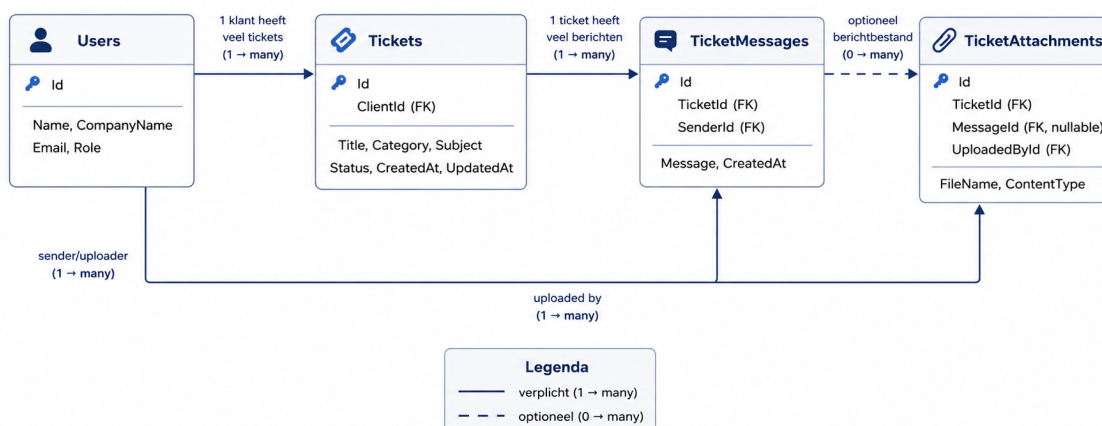
6

Databank en persistentie

Dit hoofdstuk bespreekt hoe data wordt bewaard. Forcebit Ticketing gebruikt een MySQL-databank met Entity Framework Core als Object-Relational Mapper (ORM). EF Core koppelt C#-entiteiten aan databankbewerkingen (Microsoft, 2026c). Daardoor kan de applicatie met domeinobjecten werken zonder overal rechtstreeks SQL te schrijven.

6.1. Datamodel

Het datamodel draait rond gebruikers, tickets, berichten en bijlagen. Een gebruiker kan meerdere tickets hebben, een ticket heeft meerdere berichten en bijlagen horen bij een ticket of specifiek bericht. De eerste beschrijving van een ticket wordt als eerste bericht bewaard. Daardoor blijft de volledige conversatie in hetzelfde model zitten.



Figuur 6.1: Databankschema van de belangrijkste tabellen en relaties.

6.2. DbContext, configuraties en repositories

AppDbContext is de centrale EF Core-klasse. De DbSet-properties tonen welke tabellen bestaan. De detailconfiguratie staat in aparte configuratieklassen, zodat de context kort blijft en elke tabelmapping apart onderhouden kan worden.

```

1 public DbSet<User> Users => Set<User>();
2 public DbSet<Ticket> Tickets => Set<Ticket>();
3 public DbSet<TicketMessage> TicketMessages => Set<TicketMessage>();
4 public DbSet<TicketAttachment> TicketAttachments => Set<TicketAttachment>();
5
6 protected override void OnModelCreating(ModelBuilder modelBuilder)
7 {
8     base.OnModelCreating(modelBuilder);
9     modelBuilder.ApplyConfigurationsFromAssembly(typeof(AppDbContext).Assembly);
10 }

```

Codefragment 6.1: AppDbContext laadt configuraties uit de Persistence-laag.

Repositories schermen querydetails af van de service-laag. TicketService vraagt bijvoorbeeld ticketdetails op, maar hoeft niet te weten welke relaties met Include geladen worden. Daardoor beschrijft de service de workflow en werkt de repository de query uit.

6.3. Enumwaarden en relaties

Ticketstatus, categorie en onderwerp zijn enums in de Domain-laag. Een enum groepeerde vaste waarden binnen een eigen type (Microsoft, 2026b). In de databank worden deze waarden met HasConversion<string>() als tekst opgeslagen. Een status zoals Open is tijdens controle duidelijker dan een numerieke waarde.

De relatie tussen ticket en klant gebruikt DeleteBehavior.Restrict. Daardoor kan een gebruiker niet zomaar verwijderd worden zolang tickets nog naar die gebruiker verwijzen. Bijlagen worden als metadata bewaard met een pad naar het bestand. Een bijlage kan aan een ticket hangen en optioneel aan een specifiek bericht. Zo blijft de ticketgeschiedenis controleerbaar.

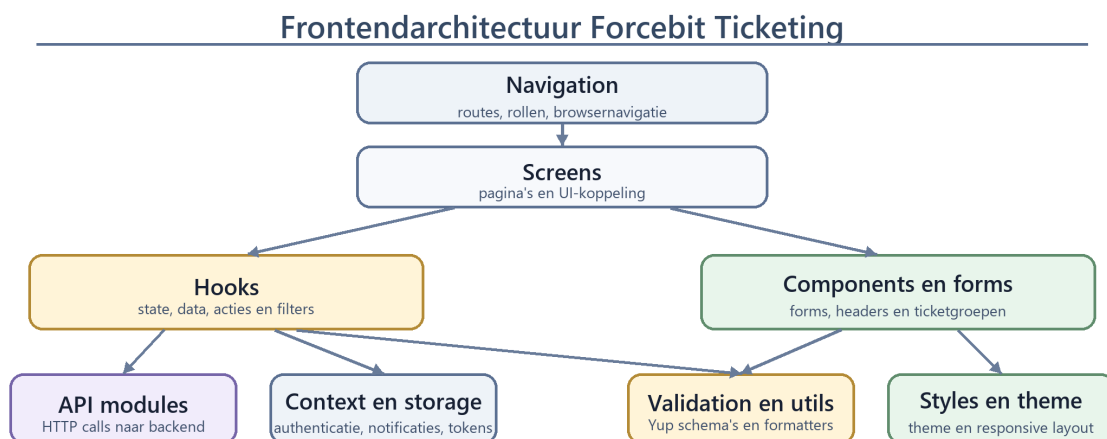
7

Frontendarchitectuur en gebruikersinterface

De frontend is gebouwd met Expo React Native en opgesplitst in schermen, hooks, componenten, API-modules, validatie, stijlen en hulpfuncties.

7.1. Schermen, hooks en componenten

Schermen bepalen wat zichtbaar is. Hooks beheren gedrag zoals laden, zoeken, filteren en pagineren. Componenten vormen herbruikbare UI-blokken. API-modules communiceren met de backend.

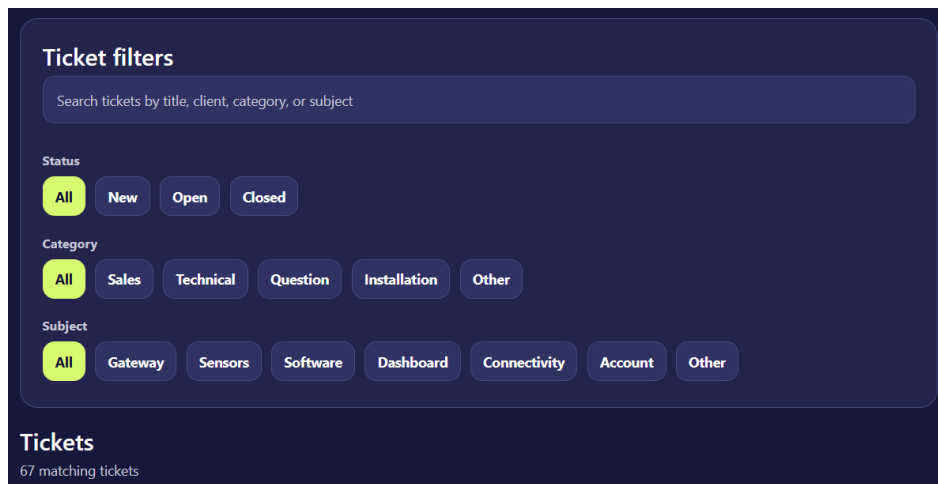


Figuur 7.1: Schema van de frontendstructuur met navigatie, schermen, hooks, herbruikbare componenten en ondersteunende modules.

7.2. Dashboardlogica

De dashboards bewaren de ruwe tickets als bron van waarheid. Daarna worden zoekresultaten, filters, statusgroepen, tellingen en pagina's afgeleid. Dit vermindert de kans op inconsistente lijsten. Als de ruwe ticketlijst verandert, worden de afgeleide resultaten opnieuw berekend.

Conceptueel vertrekt de dashboardlogica van ruwe data en loopt ze daarna via zoeken, filters, statusgroepen en paginatie.



Figuur 7.2: Ticketoverzicht met zoekveld en filters voor status, categorie en onderwerp.

7.3. Paginatie en hergebruik

Paginatie is gecentraliseerd in `usePagination`. De generieke hook werkt voor tickets, gesprekken en klantenlijsten, zodat dezelfde logica niet op meerdere plaatsen terugkomt.

```

1 export function usePagination<T>(  

2   items: T[],  

3   { pageSize, initialPage = "first", resetKey }: UsePaginationOptions,  

4 ) {  

5   const totalPages = Math.max(1, Math.ceil(items.length / pageSize));  

6   const [currentPage, setCurrentPage] = useState(1);  

7  

8   const pageItems = useMemo(  

9     () => items.slice(startIndex, endIndex),  

10    [endIndex, items, startIndex],  

11  );  

12  

13  return { pageItems, currentPage, totalPages };  

14 }

```

Codefragment 7.1: Vereenvoudigd voorbeeld van de generieke paginatiehook.

7.4. Ticketgroepen en conversatie

De klant- en adminpagina's gebruiken statusgroepen voor New, Open en Closed. TicketGroupSection toont een groep tickets en gebruikt intern dezelfde paginatiehooek. Elke groep heeft eigen paginatie.

In het ticketdetail staan adminberichten en klantberichten aan verschillende kanten met een andere achtergrondkleur. Daardoor blijft een lange conversatie overzichtelijker.

SKYFIRE TEST COMPANY Open

Installation checklist questions before rollout

Installation / Other
Created Apr 19, 2026, 06:37 AM

Status
Open Closed

Conversation

Skyfire Client Apr 19, 2026, 06:37 AM
Before the rollout starts, can you review what our team needs to prepare on site?

Alex Admin Jun 5, 2026, 03:39 AM
Yes, they have to. Here is a list of the deadlines.

Data en deadlines nbt de STAGE GRIT Academiejear 2025-26			
Akte	Semester	Start Datum	Eind Datum
Semester 1			
1. Inleiding	Week 1 tot Week 2	01-09-2025	01-10-2025
2. Het proces	Week 3 tot Week 4	02-10-2025	01-11-2025
3. Het proces	Week 5 tot Week 6	02-11-2025	01-12-2025
4. Het proces	Week 7 tot Week 8	02-12-2025	01-01-2026
5. Het proces	Week 9 tot Week 10	02-01-2026	01-02-2026
6. Het proces	Week 11 tot Week 12	02-02-2026	01-03-2026
7. Het proces	Week 13 tot Week 14	02-03-2026	01-04-2026
8. Het proces	Week 15 tot Week 16	02-04-2026	01-05-2026
9. Het proces	Week 17 tot Week 18	02-05-2026	01-06-2026
10. Het proces	Week 19 tot Week 20	02-06-2026	01-07-2026
11. Het proces	Week 21 tot Week 22	02-07-2026	01-08-2026
12. Het proces	Week 23 tot Week 24	02-08-2026	01-09-2026
13. Het proces	Week 25 tot Week 26	02-09-2026	01-10-2026
14. Het proces	Week 27 tot Week 28	02-10-2026	01-11-2026
15. Het proces	Week 29 tot Week 30	02-11-2026	01-12-2026
16. Het proces	Week 31 tot Week 32	02-12-2026	01-01-2027
17. Het proces	Week 33 tot Week 34	02-01-2027	01-02-2027
18. Het proces	Week 35 tot Week 36	02-02-2027	01-03-2027
19. Het proces	Week 37 tot Week 38	02-03-2027	01-04-2027
20. Het proces	Week 39 tot Week 40	02-04-2027	01-05-2027
21. Het proces	Week 41 tot Week 42	02-05-2027	01-06-2027
22. Het proces	Week 43 tot Week 44	02-06-2027	01-07-2027
23. Het proces	Week 45 tot Week 46	02-07-2027	01-08-2027
24. Het proces	Week 47 tot Week 48	02-08-2027	01-09-2027
25. Het proces	Week 49 tot Week 50	02-09-2027	01-10-2027
26. Het proces	Week 51 tot Week 52	02-10-2027	01-11-2027
27. Het proces	Week 53 tot Week 54	02-11-2027	01-12-2027
28. Het proces	Week 55 tot Week 56	02-12-2027	01-01-2028
29. Het proces	Week 57 tot Week 58	02-01-2028	01-02-2028
30. Het proces	Week 59 tot Week 60	02-02-2028	01-03-2028
31. Het proces	Week 61 tot Week 62	02-03-2028	01-04-2028
32. Het proces	Week 63 tot Week 64	02-04-2028	01-05-2028
33. Het proces	Week 65 tot Week 66	02-05-2028	01-06-2028
34. Het proces	Week 67 tot Week 68	02-06-2028	01-07-2028
35. Het proces	Week 69 tot Week 70	02-07-2028	01-08-2028
36. Het proces	Week 71 tot Week 72	02-08-2028	01-09-2028
37. Het proces	Week 73 tot Week 74	02-09-2028	01-10-2028
38. Het proces	Week 75 tot Week 76	02-10-2028	01-11-2028
39. Het proces	Week 77 tot Week 78	02-11-2028	01-12-2028
40. Het proces	Week 79 tot Week 80	02-12-2028	01-01-2029
41. Het proces	Week 81 tot Week 82	02-01-2029	01-02-2029
42. Het proces	Week 83 tot Week 84	02-02-2029	01-03-2029
43. Het proces	Week 85 tot Week 86	02-03-2029	01-04-2029
44. Het proces	Week 87 tot Week 88	02-04-2029	01-05-2029
45. Het proces	Week 89 tot Week 90	02-05-2029	01-06-2029
46. Het proces	Week 91 tot Week 92	02-06-2029	01-07-2029
47. Het proces	Week 93 tot Week 94	02-07-2029	01-08-2029
48. Het proces	Week 95 tot Week 96	02-08-2029	01-09-2029
49. Het proces	Week 97 tot Week 98	02-09-2029	01-10-2029
50. Het proces	Week 99 tot Week 100	02-10-2029	01-11-2029
51. Het proces	Week 101 tot Week 102	02-11-2029	01-12-2029
52. Het proces	Week 103 tot Week 104	02-12-2029	01-01-2030
53. Het proces	Week 105 tot Week 106	02-01-2030	01-02-2030
54. Het proces	Week 107 tot Week 108	02-02-2030	01-03-2030
55. Het proces	Week 109 tot Week 110	02-03-2030	01-04-2030
56. Het proces	Week 111 tot Week 112	02-04-2030	01-05-2030
57. Het proces	Week 113 tot Week 114	02-05-2030	01-06-2030
58. Het proces	Week 115 tot Week 116	02-06-2030	01-07-2030
59. Het proces	Week 117 tot Week 118	02-07-2030	01-08-2030
60. Het proces	Week 119 tot Week 120	02-08-2030	01-09-2030
61. Het proces	Week 121 tot Week 122	02-09-2030	01-10-2030
62. Het proces	Week 123 tot Week 124	02-10-2030	01-11-2030
63. Het proces	Week 125 tot Week 126	02-11-2030	01-12-2030
64. Het proces	Week 127 tot Week 128	02-12-2030	01-01-2031
65. Het proces	Week 129 tot Week 130	02-01-2031	01-02-2031
66. Het proces	Week 131 tot Week 132	02-02-2031	01-03-2031
67. Het proces	Week 133 tot Week 134	02-03-2031	01-04-2031
68. Het proces	Week 135 tot Week 136	02-04-2031	01-05-2031
69. Het proces	Week 137 tot Week 138	02-05-2031	01-06-2031
70. Het proces	Week 139 tot Week 140	02-06-2031	01-07-2031
71. Het proces	Week 141 tot Week 142	02-07-2031	01-08-2031
72. Het proces	Week 143 tot Week 144	02-08-2031	01-09-2031
73. Het proces	Week 145 tot Week 146	02-09-2031	01-10-2031
74. Het proces	Week 147 tot Week 148	02-10-2031	01-11-2031
75. Het proces	Week 149 tot Week 150	02-11-2031	01-12-2031
76. Het proces	Week 151 tot Week 152	02-12-2031	01-01-2032
77. Het proces	Week 153 tot Week 154	02-01-2032	01-02-2032
78. Het proces	Week 155 tot Week 156	02-02-2032	01-03-2032
79. Het proces	Week 157 tot Week 158	02-03-2032	01-04-2032
80. Het proces	Week 159 tot Week 160	02-04-2032	01-05-2032
81. Het proces	Week 161 tot Week 162	02-05-2032	01-06-2032
82. Het proces	Week 163 tot Week 164	02-06-2032	01-07-2032
83. Het proces	Week 165 tot Week 166	02-07-2032	01-08-2032
84. Het proces	Week 167 tot Week 168	02-08-2032	01-09-2032
85. Het proces	Week 169 tot Week 170	02-09-2032	01-10-2032
86. Het proces	Week 171 tot Week 172	02-10-2032	01-11-2032
87. Het proces	Week 173 tot Week 174	02-11-2032	01-12-2032
88. Het proces	Week 175 tot Week 176	02-12-2032	01-01-2033
89. Het proces	Week 177 tot Week 178	02-01-2033	01-02-2033
90. Het proces	Week 179 tot Week 180	02-02-2033	01-03-2033
91. Het proces	Week 181 tot Week 182	02-03-2033	01-04-2033
92. Het proces	Week 183 tot Week 184	02-04-2033	01-05-2033
93. Het proces	Week 185 tot Week 186	02-05-2033	01-06-2033
94. Het proces	Week 187 tot Week 188	02-06-2033	01-07-2033
95. Het proces	Week 189 tot Week 190	02-07-2033	01-08-2033
96. Het proces	Week 191 tot Week 192	02-08-2033	01-09-2033
97. Het proces	Week 193 tot Week 194	02-09-2033	01-10-2033
98. Het proces	Week 195 tot Week 196	02-10-2033	01-11-2033
99. Het proces	Week 197 tot Week 198	02-11-2033	01-12-2033
100. Het proces	Week 199 tot Week 200	02-12-2033	01-01-2034

Deadlines Stage.png

Reply

Here is the nessecary information you might need

Add attachments **Clear files**

Upload limit: 20 MB per reply.
Selected: 1.4 MB of 20 MB. Remaining: 19 MB.

1 file selected

Infosessie_Opstart_Stage_Studenten_2025-2026_-_Dag_-_V1.2.pdf (1.4 MB)

Send reply

Figuur 7.3: Ticketdetail met chatbubbels, visueel onderscheid tussen klant en admin en afbeeldings-preview.

8

Aanvullende functionaliteiten

Dit hoofdstuk bespreekt e-mailnotificaties, bijlagen en documentatie. De gewone README beschrijft hoe het project gestart wordt; de technische README licht architectuur, workflows en keuzes toe.

8.1. E-mailnotificaties met Brevo

Wanneer een administrator antwoordt of een status wijzigt, krijgt de klant een e-mail. De backend gebruikt hiervoor Brevo, een provider voor transactionele berichten (Brevo, [2026](#)). TicketService beslist wanneer een notificatie nodig is; EmailService bouwt en verstuurt de e-mail. Zonder Brevo API-sleutel wordt alleen een waarschuwing gelogd, zodat lokale ontwikkeling blijft werken.

8.2. Bijlagen en uploadlimiet

Bijlagen kunnen toegevoegd worden bij ticketcreatie en bij antwoorden. De frontend gebruikt Expo DocumentPicker en toont hoeveel van de 20 MB selectielimiet al gebruikt is. De backend controleert bestandstype en grootte opnieuw. Bestanden worden lokaal op de API-server opgeslagen; de databank bewaart metadata zoals bestandsnaam, contenttype, relatief pad, ticket en bericht. Afbeeldingen krijgen een preview in de conversatie, zodat screenshots meteen bekeken kunnen worden.

9

Conclusie

De onderzoeksvraag was hoe een onderhoudbare ticketingapplicatie gebouwd kan worden waarin klant en administrator een duidelijke supportworkflow volgen. Het antwoord is dat onderhoudbaarheid vooral ontstaat door verantwoordelijkheden consequent te scheiden. Controllers behandelen HTTP, services voeren use cases uit, repositories schermen databanktoegang af en domeinregels bewaken de businessregels. DTO's vormen de grens tussen requestdata en applicatielogica.

Functioneel levert Forcebit Ticketing het beoogde proof of concept op. Klanten kunnen tickets aanmaken, beantwoorden, sluiten en opnieuw openen. Administrators kunnen tickets opvolgen, zoeken, filteren, beantwoorden en van status veranderen. De statusflow met **New**, **Open** en **Closed** maakt het verschil tussen een nieuwe vraag, een lopende conversatie en een afgesloten ticket duidelijk.

De frontend volgt dezelfde lijn. Schermen tonen pagina's, hooks beheren gedrag, API-modules communiceren met de backend en componenten houden herbruikbare UI apart. Daardoor konden chatweergave, afbeeldingspreview, uploadlimiet, zoekvelden, filters en paginatie toegevoegd worden zonder alle logica in de schermen te plaatsen. Documentatie legt de belangrijkste verantwoordelijkheden en uitbreidingspunten vast.

De meerwaarde voor Forcebit is dat vraag, antwoord, status en bijlagen op een plaats samenkomen in plaats van verspreid te blijven over telefoon en losse e-mails. Voor mij als student bracht het project backend, databank, authenticatie, frontend, uploads, e-mail en documentatie samen in een realistische workflow.

Kritisch bekeken blijft het resultaat een proof of concept en geen productieklare applicatie. Voor dagelijks gebruik zijn tests, logging, secret management, productiegerichte opslag en verfijnd adminbeheer nodig. Een aparte bestandsserver of object storage past beter bij langdurig gebruik. Binnen de scope toont het project dat gescheiden verantwoordelijkheden de onderhoudbaarheid haalbaar maken.

Bibliografie

- Atlassian. (2026). *IT Ticketing Systems: Simplify Issue Tracking & Resolution*. Verkregen 5 juni 2026, van <https://www.atlassian.com/itsm/service-request-management/it-ticketing-system>
- AXELOS. (2019). *ITIL Foundation: ITIL 4 Edition*. The Stationery Office.
- Brevo. (2026). *Brevo API Documentation*. Verkregen 5 juni 2026, van <https://developers.brevo.com/docs/getting-started>
- Expo. (2026). *Expo documentation*. Verkregen 5 juni 2026, van <https://docs.expo.dev/>
- Formik. (2026). *Formik documentation*. Verkregen 5 juni 2026, van <https://formik.org/>
- Fowler, M. (2003). *Data Transfer Object*. Verkregen 5 juni 2026, van <https://martinfowler.com/eaCatalog/dataTransferObject.html>
- ISO/IEC. (2018). *ISO/IEC 20000-1:2018: Service management system requirements* (International Standard ISO/IEC 20000-1:2018). International Organization for Standardization. Verkregen 5 juni 2026, van <https://www.iso.org/standard/70636.html>
- Jacobson, I., Christerson, M., Jonsson, P. & Overgaard, G. (1992). *Object-Oriented Software Engineering: A Use Case Driven Approach*. ACM Press; Addison-Wesley.
- Jones, M. B., Bradley, J. & Sakimura, N. (2015). *JSON Web Token (JWT)*. Verkregen 4 juni 2026, van <https://www.rfc-editor.org/rfc/rfc7519>
- Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- Meta Open Source. (2026). *React Native documentation*. Verkregen 4 juni 2026, van <https://reactnative.dev/docs/getting-started>
- Microsoft. (2026a). *ASP.NET Core documentation*. Verkregen 4 juni 2026, van <https://learn.microsoft.com/aspnet/core/>
- Microsoft. (2026b). *C# Enumerations*. Verkregen 5 juni 2026, van <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/types/enums>
- Microsoft. (2026c). *Entity Framework Core documentation*. Verkregen 4 juni 2026, van <https://learn.microsoft.com/ef/core/>
- Microsoft. (2026d). *Options pattern in .NET*. Verkregen 5 juni 2026, van <https://learn.microsoft.com/en-us/dotnet/core/extensions/options>

OWASP Foundation. (2026). *Authorization Cheat Sheet*. Verkregen 4 juni 2026, van https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

Quense, J. (2026). *Yup documentation*. Verkregen 5 juni 2026, van <https://github.com/jquense/yup>

React Navigation. (2026). *React Navigation documentation*. Verkregen 4 juni 2026, van <https://reactnavigation.org/docs/getting-started>